

CSCI 423

Introduction to Algorithms



DR. RAAFAT ELFOULY



Chapter 4

SORTING ALGORITHMS

Chapter Outline

3

- Bubble sort
- Insertion sort
- Merge sort
- Quicksort

Prerequisites

4

- Before beginning this chapter, you should be able to:
 - Read and create iterative and algorithms
 - Use summations and probabilities presented in Chapter 1
 - Solve recurrence relations in Chapter 2
 - Describe growth rates and order

Goals

5

- At the end of this chapter you should be able to:
 - Explain insertion sort and its analysis
 - Explain bubble sort and its analysis

Goals (continued)

6

- Explain quicksort and its analysis

Bubble Sort

7

- If we compare pairs of adjacent elements and none are out of order, the list is sorted
- If any are out of order, we must have to swap them to get an ordered list
- Bubble sort will make passes though the list swapping any adjacent elements that are out of order

Bubble Sort

8

- After the first pass, we know that the largest element must be in the correct place
- After the second pass, we know that the second largest element must be in the correct place
- Because of this, we can shorten each successive pass of the comparison loop

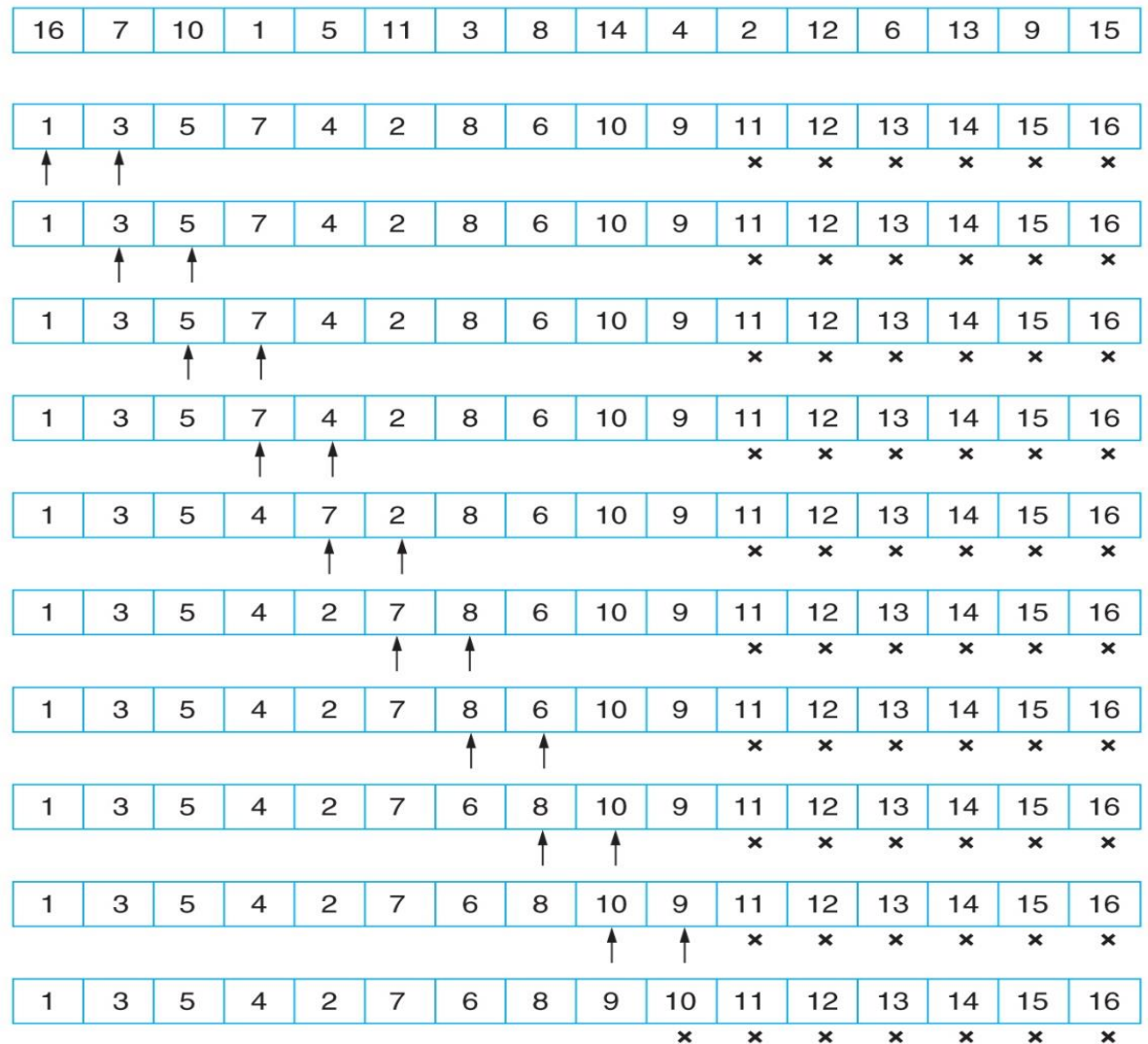


- <https://visualgo.net/bn/sorting>

Bubble Sort Example

10

■ **FIGURE 4.2**
Bubble sort





- Try to write algorithm's pseudocode

Bubble Sort Algorithm

12

```
numberOfPairs = N
swappedElements = true
while swappedElements do
  numberOfPairs = numberOfPairs - 1
  swappedElements = false
  for i = 1 to numberOfPairs do
    if list[ i ] > list[ i + 1 ] then
      Swap( list[i], list[i + 1] )
      swappedElements = true
    end if
  end for
end while
```



- Trace the algorithm for the list:
- 4, 8, 3

Best-Case Analysis

14

- If the elements start in sorted order, the for loop will compare the adjacent pairs but not make any changes
- So the `swappedElements` variable will still be false and the `while` loop is only done once
- There are $N - 1$ comparisons in the best case

Worst-Case Analysis

15

- If in the best case the `while` loop is done once, in the worst case the `while` loop must be done as many times as possible
- Each pass of the `for` loop must make at least one swap of the elements
- The number of comparisons will be:

$$W(N) = \sum_{i=N-1}^1 i = \frac{(N-1) * N}{2} = O(N^2)$$

Average-Case Analysis

16

- We can potentially stop after any of the (at most) $N - 1$ passes of the `for` loop
- This means that we have $N - 1$ possibilities and the average case is given by

$$A(N) = \frac{1}{N-1} \sum_{i=1}^{N-1} C(i)$$

- where $C(i)$ is the work done in the first i passes

Average-Case Analysis

17

- On the first pass, we do $N - 1$ comparisons
- On the second pass, we do $N - 2$ comparisons
- The number of comparisons in the first i passes, in other words $C(i)$, is given by:

$$C(i) = \sum_{j=N-1}^{N-i} j = N * i - \frac{i^2 + i}{2}$$



- $(N-i) + (N-i-1) + (N-i-2) + \dots + (N-1)$
- i terms
- First part $N*i$
- Second part : $\sum_{j=1}^i j = \frac{i^2 + i}{2}$
- Hence we get work done at stage i

$$C(i) = \sum_{j=N-1}^{N-i} j = N*i - \frac{i^2 + i}{2}$$

Average-Case Analysis

19

- Putting the equation for $C(i)$ into the equation for $A(N)$ we get:

$$\begin{aligned} A(N) &= \frac{1}{N-1} \sum_{i=1}^{N-1} \left(N * i - \frac{i^2 + i}{2} \right) \\ &= \frac{2N^2 - N}{6} \\ &= O(N^2) \end{aligned}$$



- Bubble sort in place? Needs extra memory?

Trace



Original List:	7	3	9	4	2	5	6	1	8
Pass 1:	3	7	4	2	5	6	1	8	9

```
numberOfPairs = N
swappedElements = true
while swappedElements do
  numberOfPairs = numberOfPairs - 1
  swappedElements = false
  for i = 1 to numberOfPairs do
    if list[ i ] > list[ i + 1 ] then
      Swap( list[i], list[i + 1] )
      swappedElements = true
    end if
  end for
end while
```

Insertion Sort

22

- Adding a new element to a sorted list will keep the list sorted if the element is inserted in the correct place
- A one element list is sorted
- Inserting a second element in the proper place keeps the list sorted
- This is repeated until all the elements have been inserted into the sorted part of the list

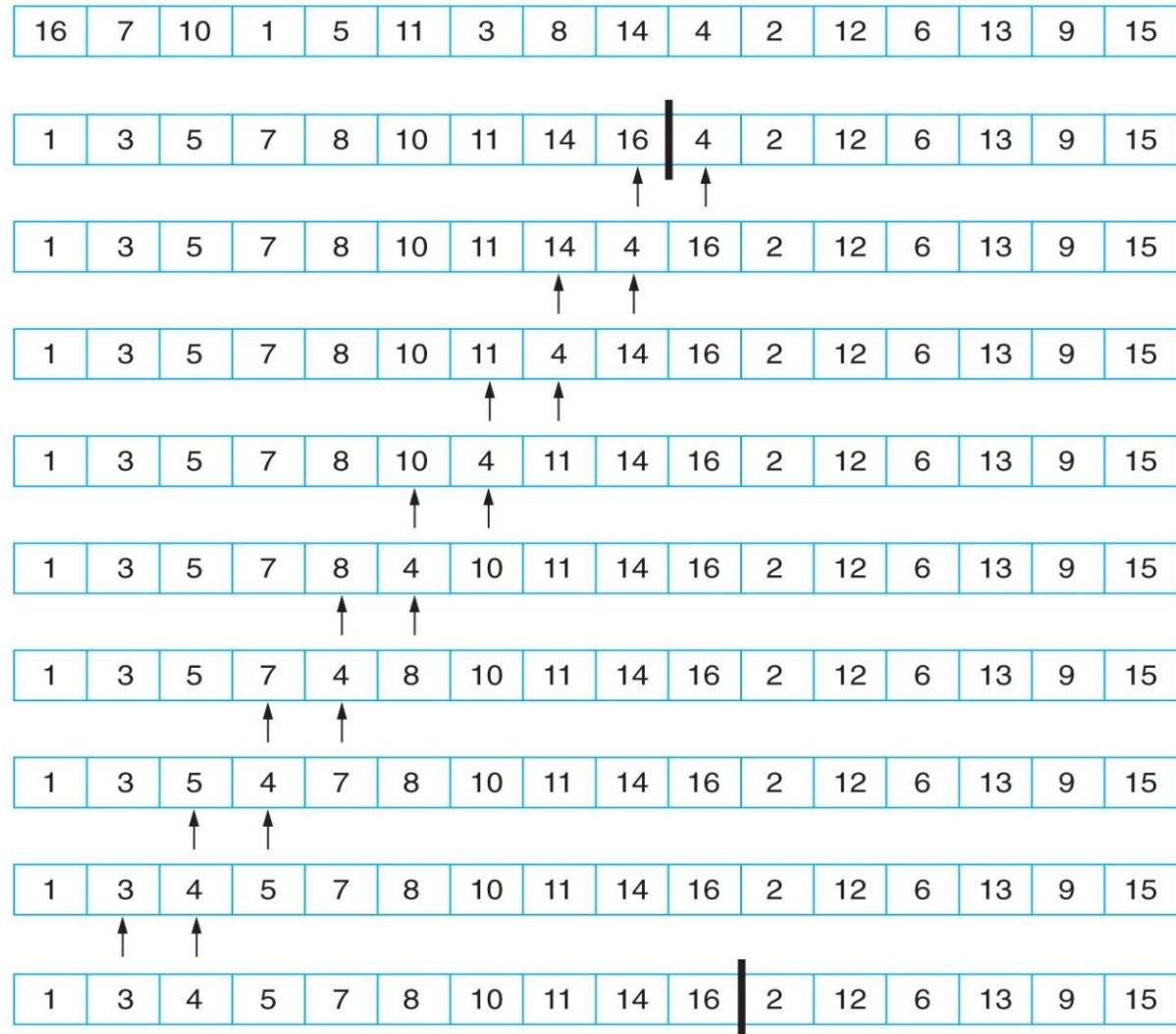


- <https://visualgo.net/bn/sorting>

Insertion Sort Example

24

■ **FIGURE 4.1**
Insertion sort



Insertion Sort

25

To sort array $A[0..n-1]$, sort $A[0..n-2]$ recursively and then insert $A[n-1]$ in its proper place among the sorted $A[0..n-2]$

- Usually implemented bottom up (nonrecursively)

Example: Sort 6, 4, 1, 8, 5

6		<u>4</u>	1	8	5
4	6		<u>1</u>	8	5
1	4	6		<u>8</u>	5
1	4	6	8		<u>5</u>
1	4	5	6	8	



- Try to write algorithm's pseudocode

Insertion Sort Algorithm

27

```
for i = 2 to N do
  newElement = list[ i ]
  location = i - 1
  while (location ≥ 1) and
    (list[ location ] > newElement) do
    list[ location + 1 ] = list[ location ]
    location = location - 1
  end while
  list[ location + 1 ] = newElement
end for
```

Best-Case Analysis

28

- If the elements start in sorted order no changes
- There are $N - 1$ comparisons in the best case

Worst-Case Analysis

29

Will happen if the list is in reverse order

- The outer loop is always done $N - 1$ times
- The inner loop does the most work when the next element is smaller than the past elements
- On each pass, the next element is compared to all earlier elements, giving:

$$W(N) = \sum_{i=1}^{N-1} i = \frac{(N-1) * N}{2} = O(N^2)$$

Average-Case Analysis

30

- The average number of comparisons to insert the i^{th} element is:

$$A_i = \frac{1}{i+1} \left[i + \sum_{p=1}^i p \right] = \frac{i}{2} + 1 - \frac{1}{i+1}$$

- We now apply this for each of the algorithm's passes:

$$A(N) = \sum_{i=1}^{N-1} A_i \approx \frac{N^2}{4} = O(N^2)$$

Insertion Sort Algorithm

final version

31

Original List:	7	3	9	4	2	5	6	1	8
Pass 1:	3	7	9	4	2	5	6	1	8

```
for i = 2 to N do
    newElement = list[ i ]
    location = i - 1
    while (location ≥ 1) and
        (list[ location ] > newElement) do
        list[ location + 1 ] = list[ location ]
        location = location - 1
    end while
    list[ location + 1 ] = newElement
end for
```

Analysis of Insertion Sort

32

- Time efficiency

$$C_{worst}(n) = n(n-1)/2 \in \Theta(n^2)$$

$$C_{avg}(n) \approx n^2/4 \in \Theta(n^2)$$

$$C_{best}(n) = n - 1 \in \Theta(n) \quad (\text{also fast on almost sorted arrays})$$

- Space efficiency: in-place
- Stability: yes
- Best elementary sorting algorithm overall
- What about Binary search would it help?

Optional HW 4 Due October 30th

(Team HW) (extra credit)



- Add binary search to insertion sort to reduce overall execution time.
- Update the algorithm and write the code for it.
- Test the code for list sizes 10K, 100K and 1 M element.
- Draw the execution time curve for the original insertion sort and the modified one (with binary search) as a function of input size(s)
- Deliverables:
 - In class demo
 - Printed code and pseudo code
 - Printed graph



Mergesort

- Old but still in use (40's)
- First divide and conquer sorting algorithm
- Split array $A[0..n-1]$ in two about equal halves and make copies of each half in arrays B and C
- Sort arrays B and C recursively

Mergesort

- Sort arrays B and C recursively
- Merge sorted arrays B and C into array A as follows:
 - Repeat the following until no elements remain in one of the arrays:
 - compare the first elements in the remaining unprocessed portions of the arrays
 - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
 - Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.



Merge Sort

- If you have two sorted lists, you can create a combined sorted list if you merge the lists
- We know that the smallest value will be the first one in either of the two lists
- If we move the smallest value to the new list, we can repeat the process until the entire list is created

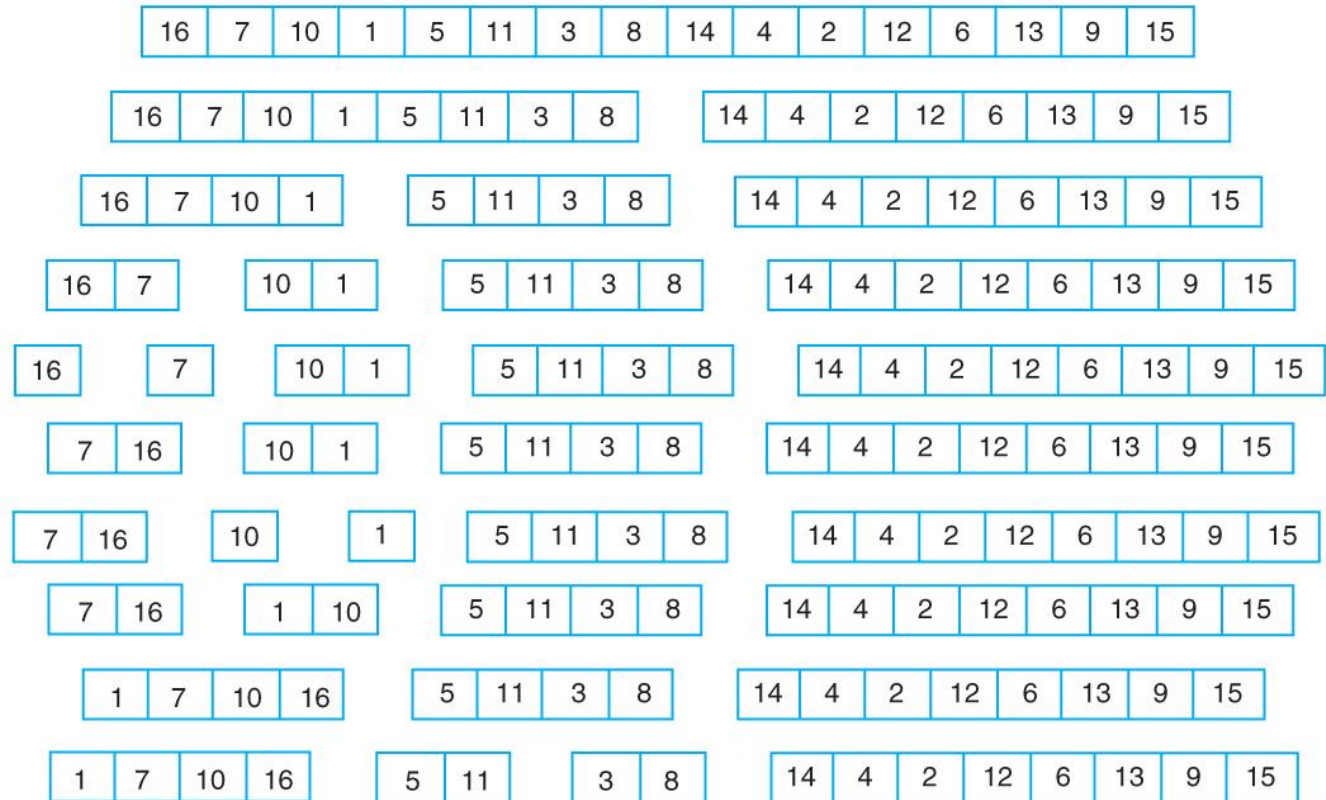


- <https://visualgo.net/bn/sorting>

Merge Sort Example

■ **FIGURE 4.8**

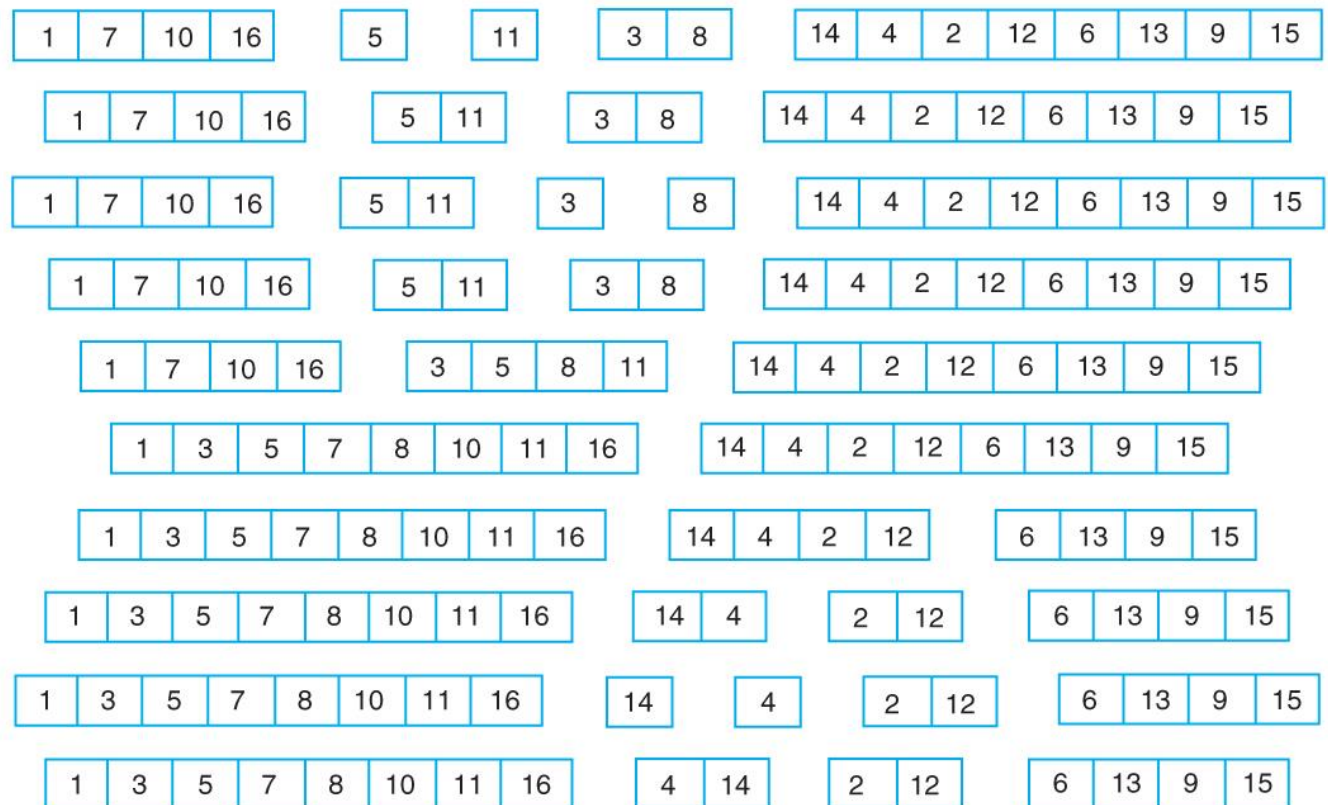
Merge sort
example



Merge Sort Example (continued)

■ **FIGURE 4.8**

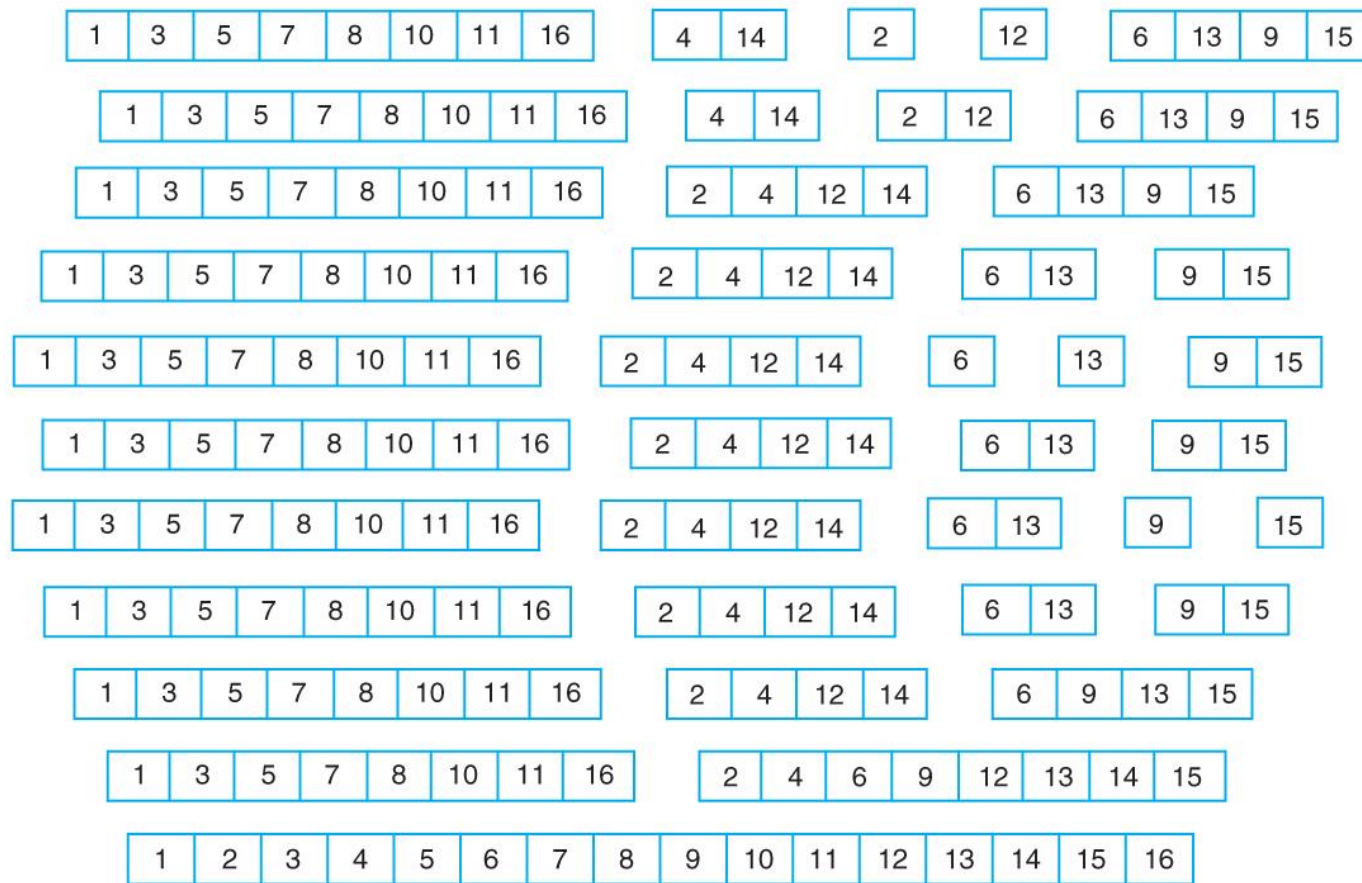
Merge sort
example



- 
- Continue the process for the second half of the list

Merge Sort Example (continued)

■ **FIGURE 4.8**
Merge sort
example



The Algorithm may look

```
if first < last then
    middle = ( first + last ) / 2
    MergeSort( list, first, middle )
    MergeSort( list, middle + 1, last )
    MergeLists( list, first, middle, middle + 1, last )
end if
```

- 
- Try to develop pseudo code for each function (Megresort, Mergelists)
 - Work in a group of 2 students
 - Test your algorithm on
 - 8 3 2 9

Pseudocode of Mergesort

ALGORITHM *Mergesort*($A[0..n - 1]$)

//Sorts array $A[0..n - 1]$ by recursive mergesort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

if $n > 1$

 copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

 copy $A[\lfloor n/2 \rfloor..n - 1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$)

Mergesort($C[0..\lceil n/2 \rceil - 1]$)

Merge(B, C, A)

Pseudocode of Merge

ALGORITHM *Merge*($B[0..p-1]$, $C[0..q-1]$, $A[0..p+q-1]$)

//Merges two sorted arrays into one sorted array

//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted

//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C

$i \leftarrow 0$; $j \leftarrow 0$; $k \leftarrow 0$

while $i < p$ **and** $j < q$ **do**

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]$; $i \leftarrow i + 1$

else $A[k] \leftarrow C[j]$; $j \leftarrow j + 1$

$k \leftarrow k + 1$

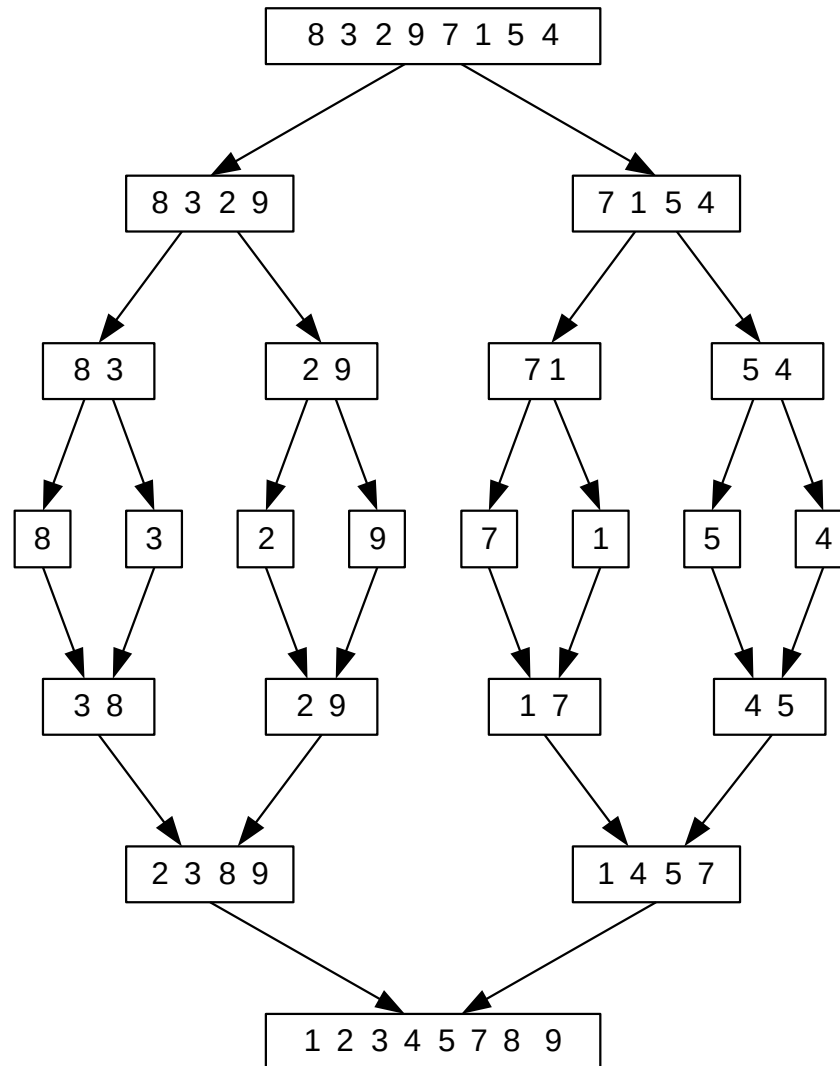
if $i = p$

 copy $C[j..q-1]$ to $A[k..p+q-1]$

else copy $B[i..p-1]$ to $A[k..p+q-1]$

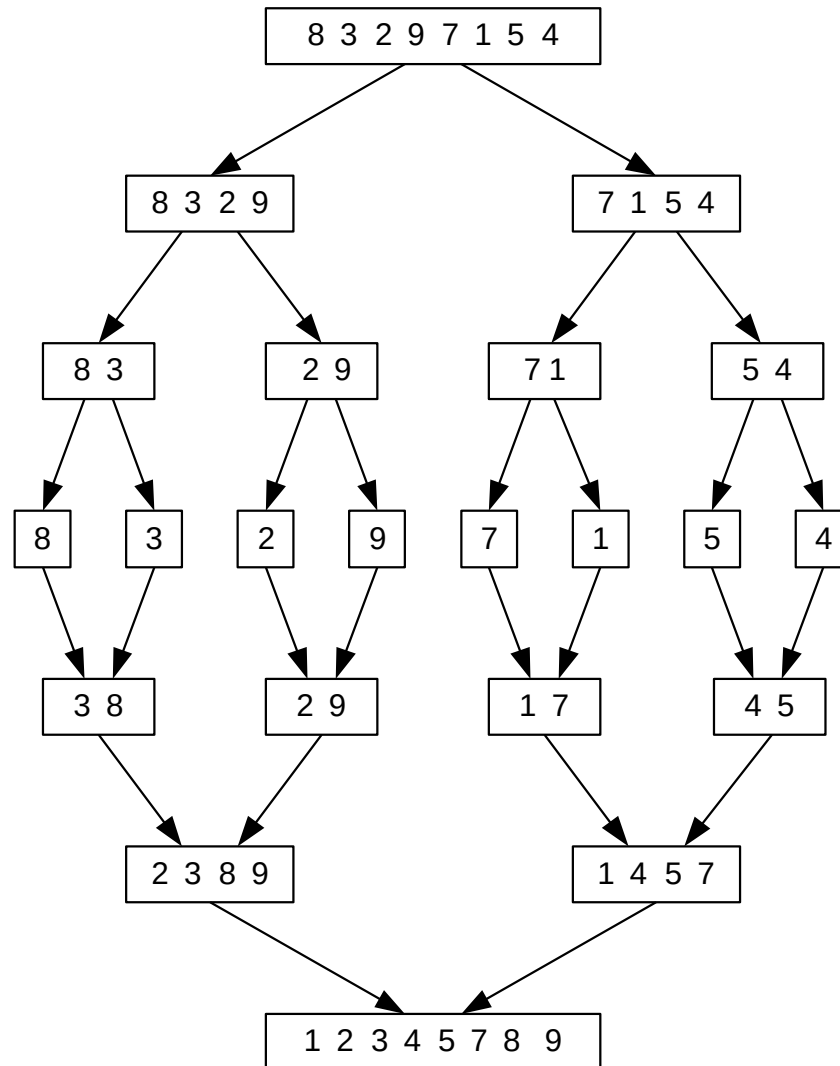
- 
- Trace
 - 8 3 2 9 7 1 5 4

Mergesort Example



Mergesort Example

Merge sort

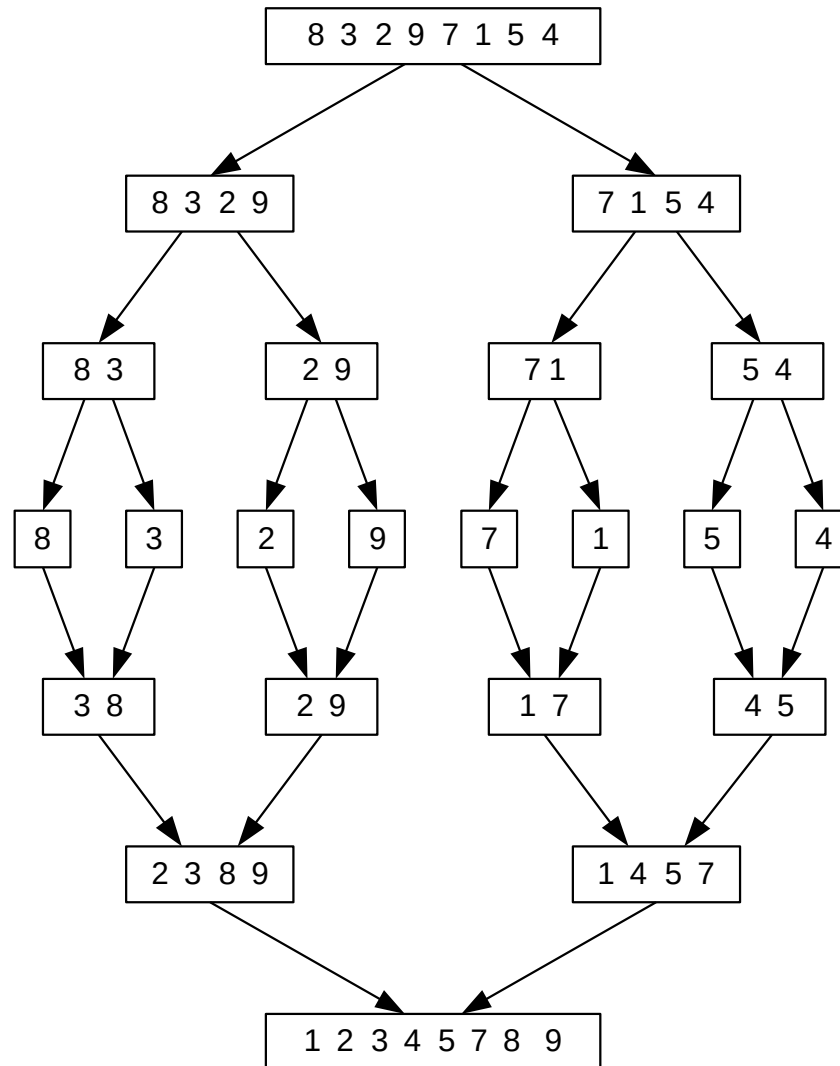


Mergesort Example

Merge sort

Merge sort

Merge sort

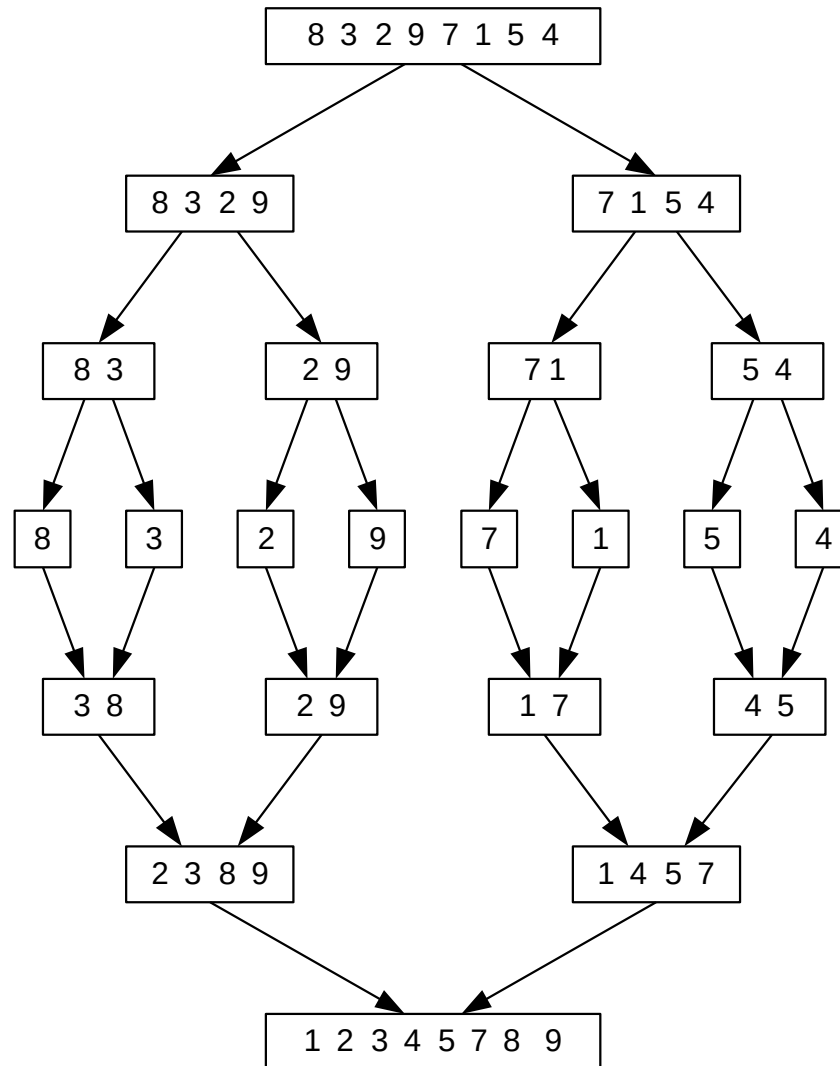


Mergesort Example

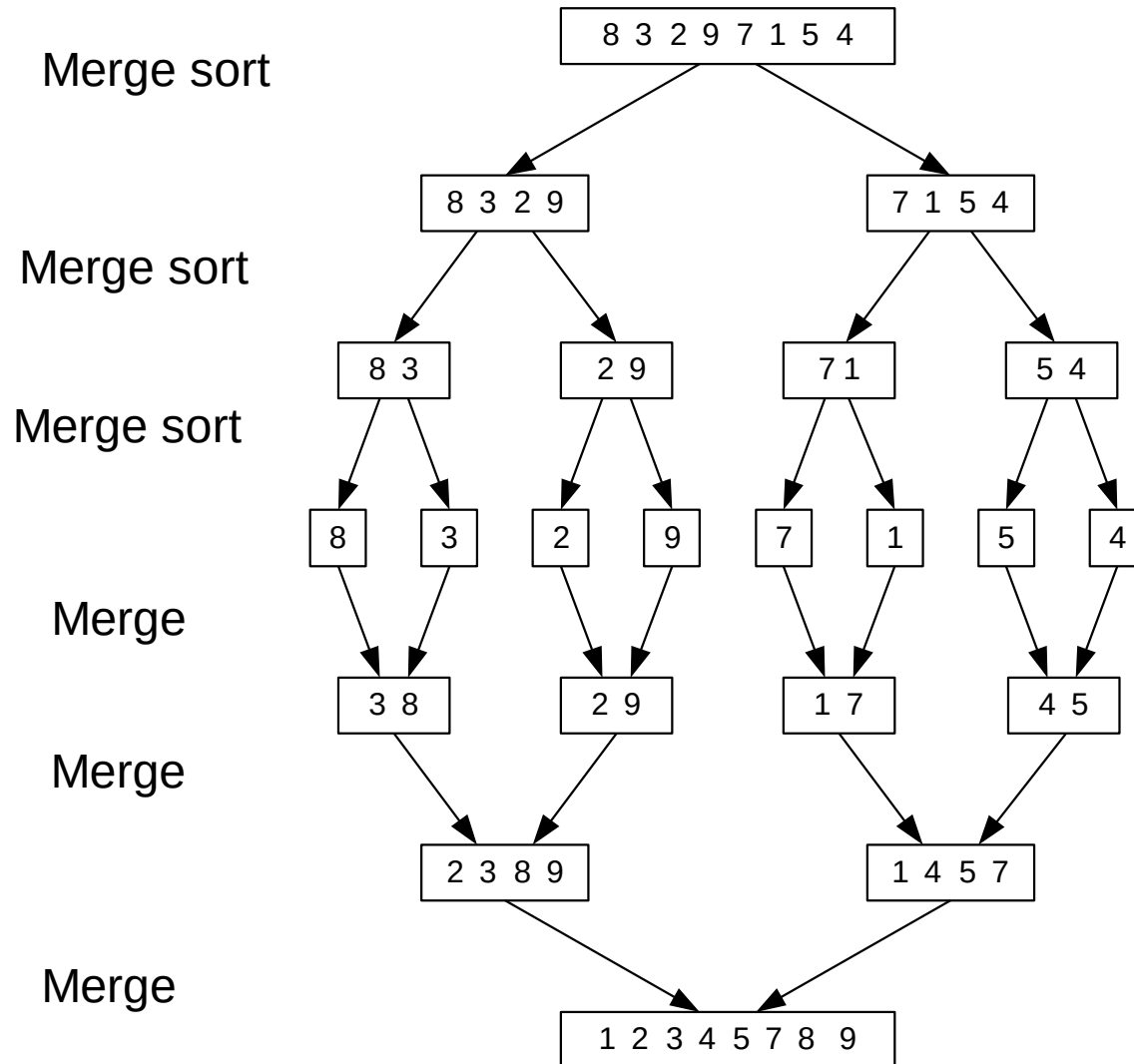
Merge sort

Merge sort

Merge sort



Mergesort Example



Mergesort Implementation

```
mergesort(item_type s[], int low, int high)
{
    int i; (* counter *)
    int middle; (* index of middle element *)

    if (low < high) {
        middle = (low+high)/2;
        mergesort(s,low,middle);
        mergesort(s,middle+1,high);

        merge(s, low, middle, high);
    }
}
```

Merging Sorted Lists

The efficiency of mergesort depends upon how efficiently we combine the two sorted halves into a single sorted list.

This smallest element can be removed, leaving two sorted lists behind, one slightly shorter than before.

Repeating this operation until both lists are empty merges two sorted lists (with a total of n elements between them) into one, using at most $n - 1$ comparisons or $O(n)$ total work

Example: $A = \{5, 7, 12, 19\}$ and $B = \{4, 6, 13, 15\}$.

MergeSort Analysis

- MergeSort divides the list in half each time, so the difference between the best and worst cases is how much work MergeList does
- In the analysis, we consider that a list of N elements gets broken into two lists of $N/2$ elements that are recursively sorted and then merged together

MergeLists Analysis

- The best case is when the elements of one list are larger than all of the elements of the other list
- One worst case is when the elements are interleaved
- If each list has N elements, we will do $N/2$ comparisons in the best case, and $N-1$ comparisons in the worst case

The Master Method

If $T(n) \leq aT\left(\frac{n}{b}\right) + O(n^d)$

then

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad (\text{Case 1}) \\ O(n^d) & \text{if } a < b^d \quad (\text{Case 2}) \\ O(n^{\log_b a}) & \text{if } a > b^d \quad (\text{Case 3}) \end{cases}$$

A decorative header with a blue abstract background featuring wavy, liquid-like patterns.

What complexity do we get?

MergeSort Analysis

- The worst case is:

$$W(N) = 2W(N/2) + N - 1$$

$$W(0) = W(1) = 0$$

which solves to $W(N) = O(N \lg N)$

MergeSort Analysis

- The worst case is:

$$W(N) = 2W(N/2) + N - 1$$

$$W(0) = W(1) = 0$$

which solves to $W(N) = O(N \lg N)$

- The best case is:

$$B(N) = 2B(N/2) + N/2$$

$$B(0) = B(1) = 0$$

which solves to $B(N) = O(N \lg N)$

Analysis of Mergesort

- All cases have same efficiency: $\Theta(n \log n)$
- Space requirement: $\Theta(N \text{ Log}N)$ (not in-place)
 - Why?

External Sorting

Which $O(n \log n)$ algorithm you use for sorting doesn't matter much until n is so big the data does not fit in memory. Mergesort proves to be the basis for the most efficient *external sorting* programs.

Disks are much slower than main memory, and benefit from algorithms that read and write data in long streams – not random access.